

Laboratorio1_AntonioVinicius

August 31, 2026

1 LAB 1 - MÉTODOS NUMÉRICOS - ANTÔNIO VINÍCIUS LIBERATO OLIVEIRA SIEBRA - 610619

1.1 COMANDOS BÁSICOS DE PYTHON

```
[1]: print('ajuste um modelo com ', 6, 'variáveis ')
```

ajuste um modelo com 6 variáveis

Colocando um ? depois da função, é possível consultar informações dela

```
[2]: print?
```

Signature: print(*args, sep=' ', end='\n',
file=None, flush=False)

Docstring:

Prints the values to a stream, or to sys.stdout by default.

sep

string inserted between values, default a space.

end

string appended after the last value, default a newline.

file

a file-like object (stream); defaults to the current sys.stdout.

flush

whether to forcibly flush the stream.

Type: builtin_function_or_method

Soma de dois números int:

```
[3]: 20 + 10
```

```
[3]: 30
```

Texto em python é no formato string. Para definir como texto, coloque entre aspas simples ou duplas

```
[5]: "metodos" + " " + "numericos"  
disciplina = "metodos numericos"  
print(disciplina)
```

metodos numericos

Juntando números numa lista chamada z

```
[6]: z = [1, 20, 50]
     z
```

```
[6]: [1, 20, 50]
```

Para chamar uma lista, usamos colchetes. A soma de uma lista concatena o valor delas. Não é feita operação matricial

```
[9]: y = [40, 91, 70]
     z+ y
```

```
[9]: [1, 20, 50, 40, 91, 70]
```

O resultado parece contraintuitivo: por que o Python não somou item a item? É porque listas guardam objetos quaisquer, e a soma de listas é uma concatenação — o mesmo comportamento que vimos em "hello" + " " + "world". Isso reflete que Python é uma linguagem de uso geral. A parte específica pra dados vem de outros pacotes, principalmente **numpy** e **pandas**.

1.2 Biblioteca Numpy para operações matemáticas mais avançadas

Importanto a biblioteca - Demos o apelido **np** ao módulo, só pra facilitar as próximas referências

```
[10]: import numpy as np
```

Em numpy, **array** é o termo genérico pra um conjunto de números com uma ou mais dimensões. Usamos **np.array()** pra criar arrays unidimensionais (vetores).

```
[11]: x = np.array([30, 420, 5])
     y = np.array([40, 91, 70])
```

Agora que x e y foram criados com **np.array()**, somá-los dá o resultado esperado.

```
[12]: x + y
```

```
[12]: array([ 70, 511,  75])
```

Em numpy, matrizes normalmente são arrays de duas dimensões, e vetores são arrays de uma dimensão. Dá pra criar um array bidimensional assim:

```
[13]: x = np.array([[-12, -1], [1, 46]])
```

O atributo **ndim** diz quantas dimensões o array tem.

```
[14]: x.ndim
```

```
[14]: 2
```

O atributo **dtype** mostra o tipo dos dados

```
[15]: x.dtype
```

```
[15]: dtype('int64')
```

`x` só tem inteiros porque só passamos inteiros pro `np.array()`. Se algum valor fosse decimal, o array inteiro viraria de ponto flutuante (`float`):

```
[16]: np.array([[29.439635, 90.590122], [76.823959, 9]]).dtype
```

```
[16]: dtype('float64')
```

A documentação mostra que dá pra forçar um array de ponto flutuante passando o argumento `dtype`:

```
[17]: np.array([[9, 7], [10, 14]], float).dtype
```

```
[17]: dtype('float64')
```

O array `x` é bidimensional. O número de linhas e colunas está no atributo `shape`.

```
[18]: x.shape
```

```
[18]: (2, 2)
```

Um **método** é uma função associada a um objeto. Por exemplo, dado um array `x`, `x.sum()` soma todos os elementos usando o método `sum()` dos arrays.

```
[19]: x = np.array([510, 99, 96, 1391])
      x.sum()
```

```
[19]: np.int64(6210)
```

Também dá pra somar passando `x` como argumento da função `np.sum()`:

```
[26]: x = np.array([-2675, -116, 153, -164])
      np.sum(x)
```

```
[26]: np.int64(-2802)
```

O método `reshape()` devolve um novo array com os mesmos elementos de `x`, mas com um formato diferente — passamos uma tupla, nesse caso `(2, 3)`, indicando 2 linhas e 3 colunas. O caractere `\n` cria uma nova linha no texto impresso.

```
[29]: x = np.array([-7, -2, -10, 6, -27, 29, 10, 4, -2, 3])
      print('beginning x:\n', x)
      x_reshape = x.reshape((2, 5))
      print('reshaped x:\n', x_reshape)
```

beginning x:

```
[ -7  -2 -10   6 -27  29  10   4  -2   3]
```

reshaped x:

```
[[ -7  -2 -10   6 -27]
 [ 29  10   4  -2   3]]
```

O resultado mostra que arrays numpy são especificados como sequência de linhas (row-major), diferente de column-major. Python (e numpy) indexa a partir do 0. Pra pegar o elemento no topo à esquerda de `x_reshape`, usamos `x_reshape[0,0]`.

```
[32]: x_reshape[0, 0]
```

```
[32]: np.int64(-7)
```

Da mesma forma, `x_reshape[1,2]` pega o elemento da segunda linha, terceira coluna:

```
[34]: x_reshape[1, 2]
```

```
[34]: np.int64(4)
```

De forma parecida, `x[2]` pega o terceiro elemento de `x`. Agora vamos modificar o elemento do topo esquerdo de `x_reshape`. Surpresa: o primeiro elemento de `x` muda junto!

```
[35]: print('x before we modify x_reshape :\n', x)
      print('x_reshape before we modify x_reshape :\n', x_reshape)
      x_reshape[0, 0] = 20
      print('x_reshape after we modify its top left element :\n', x_reshape)
      print('x after we modify top left element of x_reshape :\n', x)
```

```
x before we modify x_reshape :
[ -7  -2 -10   6 -27  29  10   4  -2   3]
x_reshape before we modify x_reshape :
[[ -7  -2 -10   6 -27]
 [ 29  10   4  -2   3]]
x_reshape after we modify its top left element :
[[ 20  -2 -10   6 -27]
 [ 29  10   4  -2   3]]
x after we modify top left element of x_reshape :
[ 20  -2 -10   6 -27  29  10   4  -2   3]
```

Modificar `x_reshape` também modificou `x` porque os dois objetos ocupam o mesmo espaço de memória. A gente acabou de ver que dá pra modificar um elemento de um array. Será que dá pra modificar uma tupla também? Não dá — e tentar gera um erro (uma *exception*).

```
[36]: my_tuple = (32, 43, 52)
      my_tuple[0] = 1
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[36], line 2
      1 my_tuple = (32, 43, 52)
----> 2 my_tuple[0] = 1
```

```
TypeError: 'tuple' object does not support item assignment
```

Alguns atributos de array: `shape` (sempre uma tupla), `ndim` (número de dimensões) e `T` (a transposta).

```
[ ]: (x_reshape.shape, x_reshape.ndim, x_reshape.T)
```

```
[ ]: ((2, 3),
      2,
      array([[5, 4],
             [2, 5],
             [3, 6]]))
```

```
[37]: np.sqrt(x)
```

```
/tmp/ipykernel_1857/393099422.py:1: RuntimeWarning: invalid value encountered in sqrt
  np.sqrt(x)
```

```
[37]: array([4.47213595,      nan,      nan, 2.44948974,      nan,
            5.38516481, 3.16227766, 2.      ,      nan, 1.73205081])
```

Também dá pra elevar ao quadrado:

```
[38]: x ** 2
```

```
[38]: array([400,   4, 100,  36, 729, 841, 100,  16,   4,   9])
```

E calcular raiz quadrada elevando a $1/2$ em vez de 2:

```
[39]: x ** 0.5
```

```
/tmp/ipykernel_1857/1523082894.py:1: RuntimeWarning: invalid value encountered in power
  x ** 0.5
```

```
[39]: array([4.47213595,      nan,      nan, 2.44948974,      nan,
            5.38516481, 3.16227766, 2.      ,      nan, 1.73205081])
```

Vamos precisar gerar dados aleatórios bastante ao longo do livro. `np.random.normal()` gera um vetor de variáveis aleatórias normais. Por padrão gera com média (`loc`) 0 e desvio padrão (`scale`) 1, e um único valor a menos que `size` seja mudado. Vamos gerar 50 variáveis independentes de uma $N(0,1)$:

```
[41]: x = np.random.normal(size=50)
      x
```

```
[41]: array([-0.44130421,  0.46757994,  0.73326195,  0.14378439,  0.8329483 ,
            -1.46845444,  1.47907449,  0.74739036, -1.79584594,  0.32784001,
```

```

1.738242 , 2.21305839, -1.32536592, -1.36380015, 1.76037817,
-0.48606411, -0.83388182, -1.77018126, -1.12295225, -0.31897953,
-0.40285017, -0.64669653, 0.09413453, -0.4835814 , -1.72096594,
-0.34912106, -0.75313472, 0.32925656, -0.9704826 , 0.90717408,
-0.41052328, -1.5253848 , -0.26442568, 0.39189575, 0.65750428,
0.44478358, -0.30437388, -1.4527467 , -0.84471515, -1.16939848,
-1.15429457, 0.42288239, 0.05502663, -0.58460955, 0.01670411,
0.06113225, 0.22306111, -0.82514369, 2.60723717, -0.67973573])

```

Criamos um array `y` somando a `x` uma variável $N(40,1)$ independente pra cada elemento:

```
[43]: y = x + np.random.normal(loc=-40, scale=1, size=50)
```

`np.corrcoef()` calcula a matriz de correlação entre `x` e `y`. Os elementos fora da diagonal dão a correlação entre eles.

```
[44]: np.corrcoef(x, y)
```

```
[44]: array([[1.          , 0.69886534],
            [0.69886534, 1.          ]])
```

Se você reparar, cada vez que chamamos `np.random.normal()` o resultado é diferente:

```
[48]: print(np.random.normal(scale=2, size=6))
      print(np.random.normal(scale=1, size=20))
```

```

[ 3.78340961  2.0381897 -4.17923455  0.66876389  1.40407941 -1.89243692]
[ 0.21355612 -1.98916153  0.38169458 -0.46188478  0.07342311 -0.26419186
  0.4016056  -0.66459155  0.17334428 -2.30382334 -0.75206256  0.27464836
 -0.21277197 -0.57459979  0.54205018  1.91255573 -0.75714757  0.39102886
 -0.68518067  2.12016077]

```

Pra garantir que o código sempre dê o mesmo resultado, fixamos uma `seed` com `np.random.default_rng()`, passando um número inteiro qualquer. O objeto `rng` criado tem praticamente todos os métodos de geração aleatória do `np.random` — por isso usamos `rng.normal()`.

```
[49]: rng = np.random.default_rng(1303)
      print(rng.normal(scale=7, size=7))
      rng2 = np.random.default_rng(1303)
      print(rng2.normal(scale=1, size=1))
```

```

[ 5.73275685 -1.50479847 -14.21510434  1.58768605 -5.79642793
 -6.64803752  0.68236175]
[0.81896526]

```

`np.mean()`, `np.var()` e `np.std()` calculam média, variância e desvio padrão de arrays. Essas mesmas contas também existem como método do array.

```
[51]: rng = np.random.default_rng(10)
      y = rng.standard_normal(15)
      (np.mean(y), y.mean())
```

```
[51]: (np.float64(-0.24514340147551975), np.float64(-0.24514340147551975))
```

```
[52]: (np.var(y), y.var(), np.mean((y - y.mean()) ** 2))
```

```
[52]: (np.float64(0.38517851632174055),  
      np.float64(0.38517851632174055),  
      np.float64(0.38517851632174055))
```

Por padrão `np.var()` divide por `n`, não por `n-1` — dá pra mudar isso com o argumento `ddof`.

```
[54]: (np.sqrt(np.var(y)), np.std(y))
```

```
[54]: (np.float64(0.6206275181795765), np.float64(0.6206275181795765))
```

`np.mean()`, `np.var()` e `np.std()` também rodam por linha ou coluna de uma matriz. Vamos criar uma matriz 10×3 de $N(0,1)$ e somar por linha.

```
[55]: X = rng.standard_normal((10, 8))  
      X
```

```
[55]: array([[ -1.13448753,  0.30570522, -1.85168503, -0.17705351,  0.42582567,  
          -0.98535561, -1.11295413, -0.76062603],  
         [ 0.64802459, -0.12983136, -1.86959723, -0.42334911,  1.0138968 ,  
          0.98371534,  0.63004195, -0.23805881],  
         [-1.84493988,  0.16957773, -0.17597776,  0.07679986,  1.54230411,  
          0.18368354,  0.27633381,  0.60509749],  
         [-0.25656891, -0.66435634, -0.73737305,  0.76696648,  0.50455269,  
          -0.48954647,  1.15269532,  0.18433148],  
         [-1.34021839,  0.60587897, -0.13948568, -1.32880182,  0.51548485,  
          -0.34481652, -0.3923426 ,  0.58989592],  
         [-2.19249673, -1.27730022, -0.42452017,  0.24916358, -0.66510728,  
          -0.99671724, -1.05242309, -0.11729315],  
         [ 0.67501993, -1.45000211, -2.08037901, -0.29737618, -0.38201813,  
          -0.90898046,  0.61174277,  0.80463159],  
         [-0.56416757, -0.34445141,  0.5700292 , -0.42921262, -1.40314438,  
          -0.06756824, -0.90423251,  0.32528005],  
         [-0.50517391, -0.18915224,  0.51941333, -0.1248191 , -0.63401622,  
          -1.0064569 ,  1.72106998,  1.10660861],  
         [-1.50365276, -1.05243543,  0.58401204, -0.58836485,  0.01516341,  
          0.81401159,  0.1287891 , -0.12921638]])
```

Como arrays são row-major, o primeiro eixo (`axis=0`) se refere às linhas.

```
[56]: X.mean(axis=0)
```

```
[56]: array([-0.80186611, -0.40263672, -0.56055634, -0.22760473,  0.09329415,  
          -0.2818031 ,  0.10587206,  0.23706508])
```

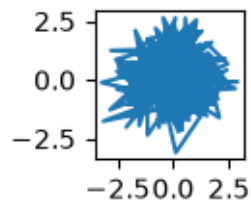
```
[57]: X.mean(0)
```

```
[57]: array([-0.80186611, -0.40263672, -0.56055634, -0.22760473,  0.09329415,  
          -0.2818031 ,  0.10587206,  0.23706508])
```

1.3 Gráficos em Python. Biblioteca matplotlib

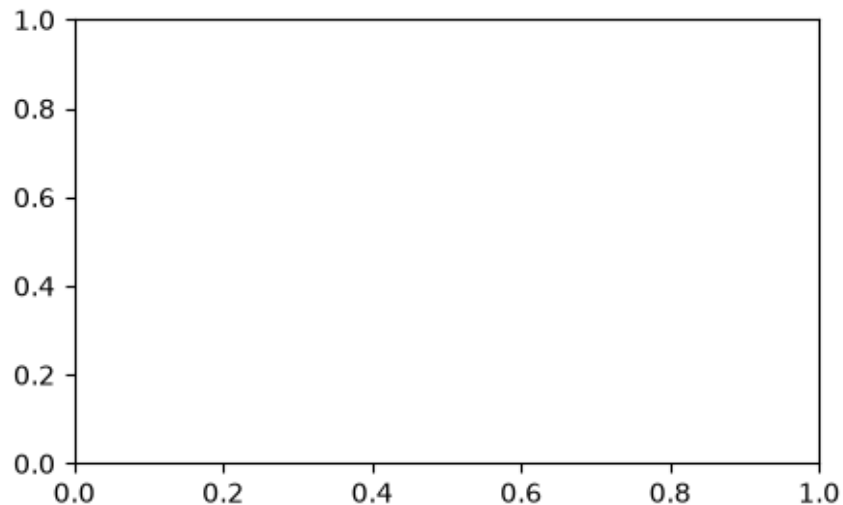
```
[65]: from matplotlib.pyplot import subplots  
fig, ax = subplots(figsize=(1, 1))  
x = rng.standard_normal(500)  
y = rng.standard_normal(500)  
ax.plot(x, y)
```

```
[65]: [<matplotlib.lines.Line2D at 0xffff334405d0>]
```



Aqui separamos a tupla de tamanho 2 devolvida por `subplots()` em `fig` e `ax`. É preferível a isso, mais verboso:

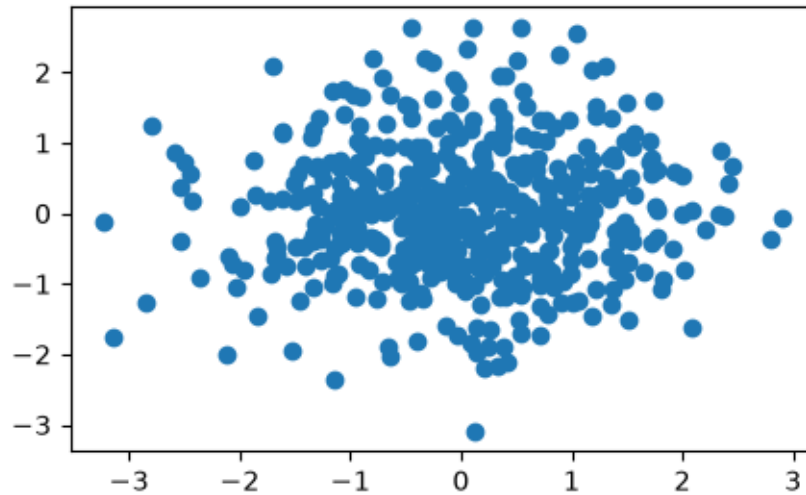
```
[69]: output = subplots(figsize=(5, 3))  
fig = output[0]  
ax = output[1]
```



A célula anterior gerou um gráfico de linha, que é o padrão. Pra virar dispersão, passamos um argumento extra pro `ax.plot()` indicando círculos:

```
[70]: fig, ax = subplots(figsize=(5, 3))  
      ax.plot(x, y, 'o')
```

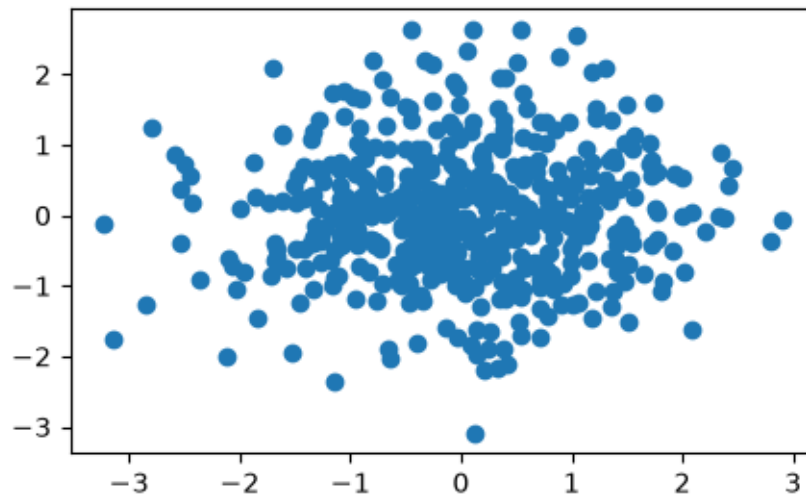
```
[70]: [<matplotlib.lines.Line2D at 0xffff32af6a10>]
```



Como alternativa, dá pra usar `ax.scatter()` direto:

```
[71]: fig, ax = subplots(figsize=(5, 3))  
      ax.scatter(x, y, marker='o')
```

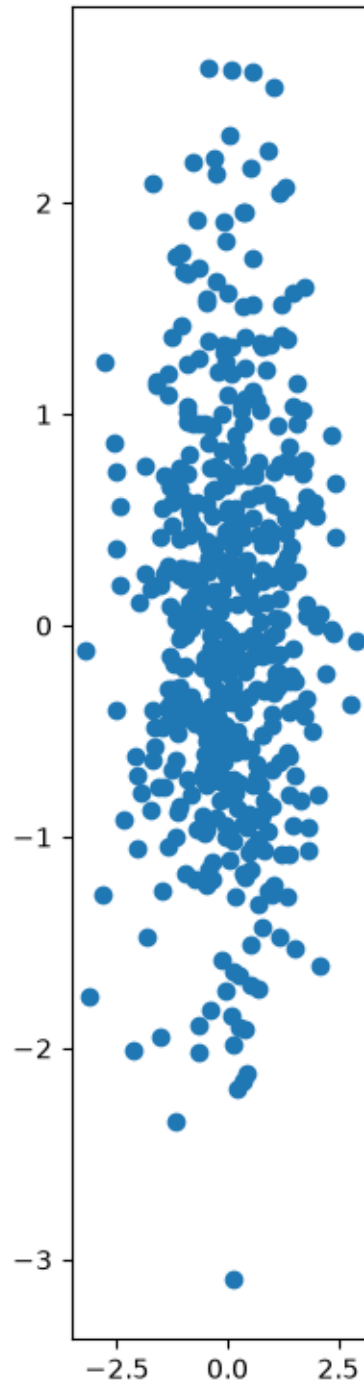
```
[71]: <matplotlib.collections.PathCollection at 0xffff38c59c10>
```



Repare que terminamos a última linha com ponto e vírgula — isso evita que o retorno de `ax.scatter(x,y)` seja impresso no notebook, mas não impede o gráfico de aparecer. Sem o `;`, o texto de retorno aparece:

```
[72]: fig, ax = subplots(figsize=(2, 9))  
      ax.scatter(x, y, marker='o')
```

```
[72]: <matplotlib.collections.PathCollection at 0xffff3208da90>
```

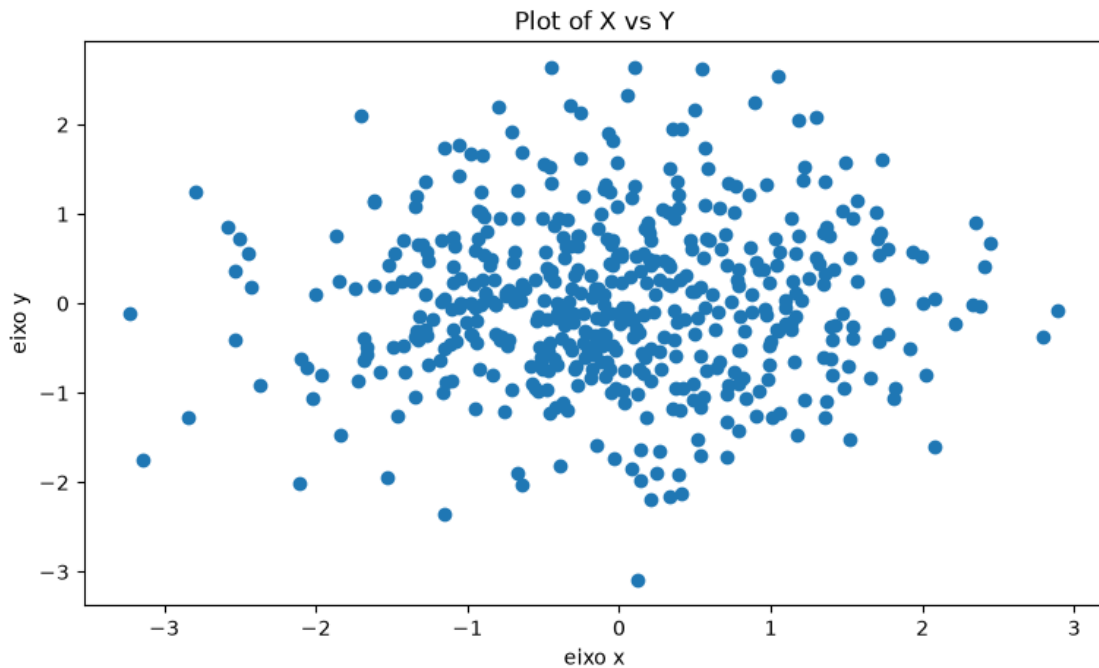


Pra colocar rótulos no gráfico, usamos `set_xlabel()`, `set_ylabel()` e `set_title()` do `ax`.

```
[73]: fig, ax = subplots(figsize=(9, 5))  
      ax.scatter(x, y, marker='o')  
      ax.set_xlabel('eixo x')
```

```
ax.set_ylabel('eixo y')
ax.set_title('Plot of X vs Y')
```

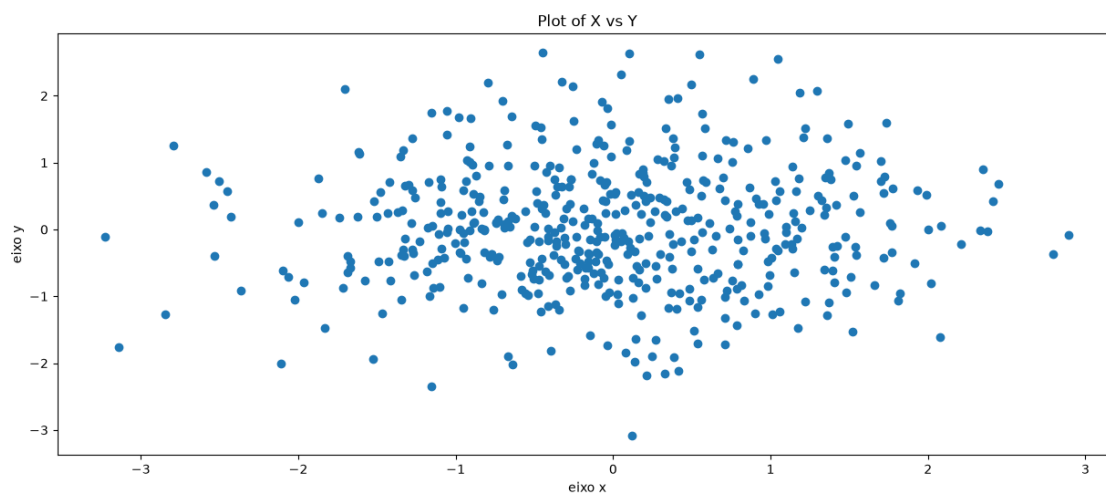
[73]: `Text(0.5, 1.0, 'Plot of X vs Y')`



Como temos acesso ao objeto `fig`, dá pra mudar coisas nele e redesenhar. Aqui mudamos o tamanho de (9,5) pra (15,6).

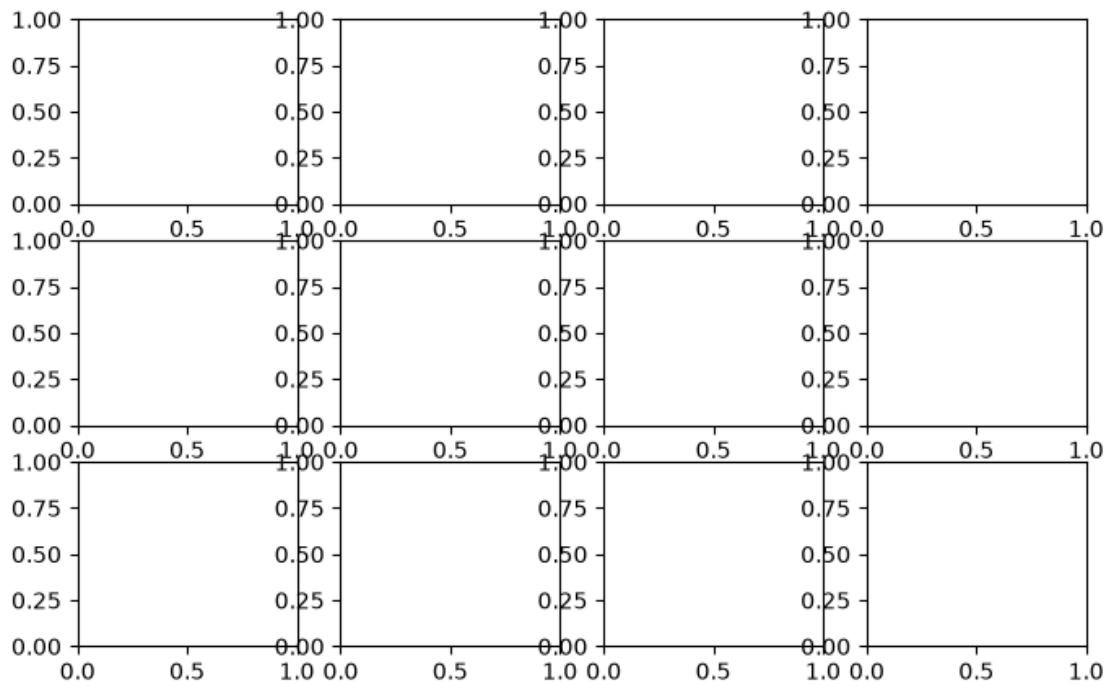
```
[74]: fig.set_size_inches(15, 6)
fig
```

[74]:



Às vezes queremos vários gráficos numa mesma figura. Dá pra fazer passando argumentos extras pro `subplots()` — abaixo, uma grade 3×4. `sharex=True` faz os plots compartilharem o eixo x.

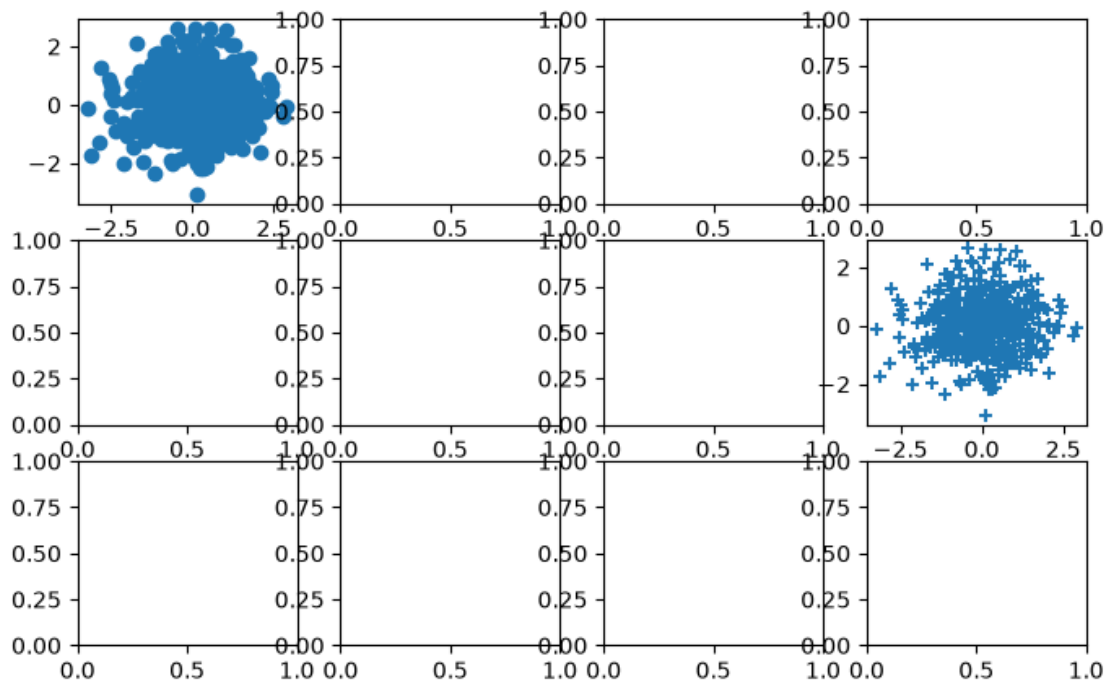
```
[76]: fig, axes = subplots(nrows=3, ncols=4, figsize=(8, 5))
```



Agora colocamos um scatter com 'o' na primeira coluna da primeira linha, e um scatter com '+' na quarta coluna da segunda linha.

```
[77]: axes[0, 0].plot(x, y, 'o')
      axes[1, 3].scatter(x, y, marker='+')
      fig
```

[77]:



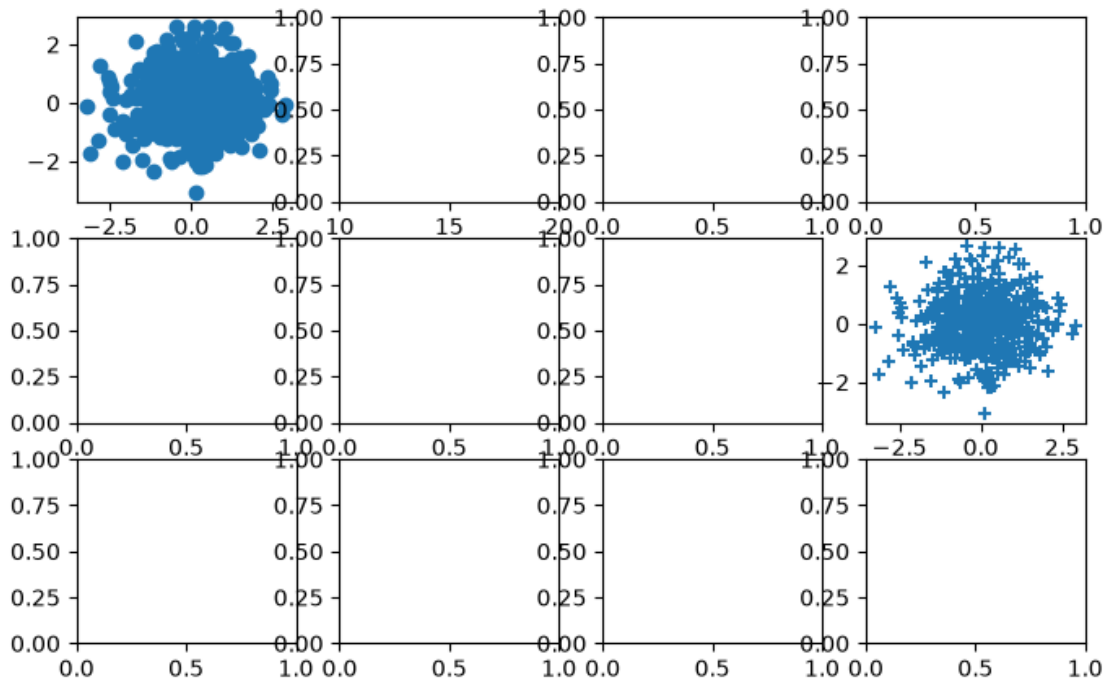
Pra salvar a figura, chamamos o método `savefig()`. O argumento `dpi` (dots per inch) define o tamanho em pixels.

```
[78]: fig.savefig('Figure.png', dpi=62)
      fig.savefig('Figure.pdf', dpi=32)
```

Dá pra continuar modificando `fig` passo a passo — por exemplo, mudar o intervalo do eixo x, re-salvar e re-exibir.

```
[79]: axes[0, 1].set_xlim([--10, 20])
      fig.savefig('Figure_updated.jpg')
      fig
```

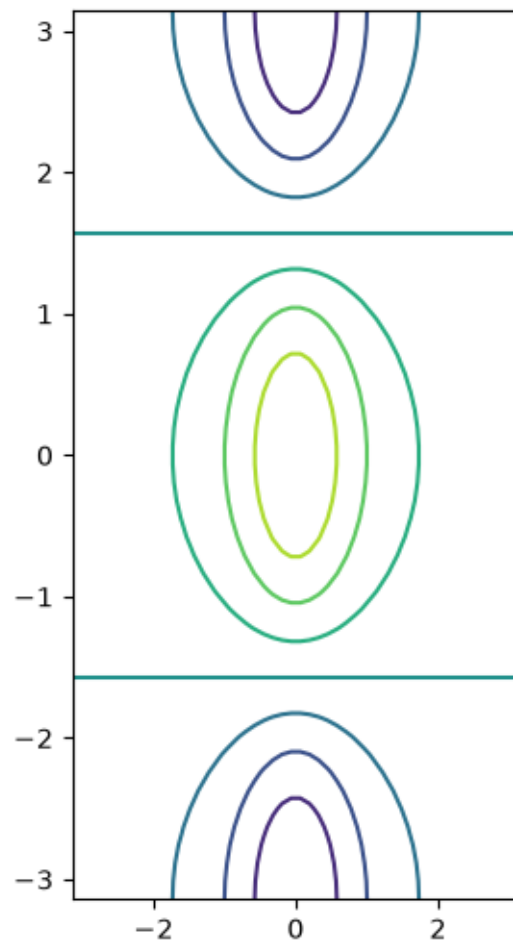
[79]:



Agora gráficos mais sofisticados. `ax.contour()` produz um gráfico de contorno pra representar dados em 3 dimensões, parecido com um mapa topográfico. Recebe três argumentos: um vetor de valores x, um vetor de valores y, e uma matriz com o valor z pra cada par (x,y). `np.linspace(a, b, n)` cria um vetor de n números entre a e b.

```
[83]: fig, ax = subplots(figsize=(3, 6))
x = np.linspace(-np.pi, np.pi, 50)
y = x
f = np.multiply.outer(np.cos(y), 1 / (1 + x ** 2))
ax.contour(x, y, f)
```

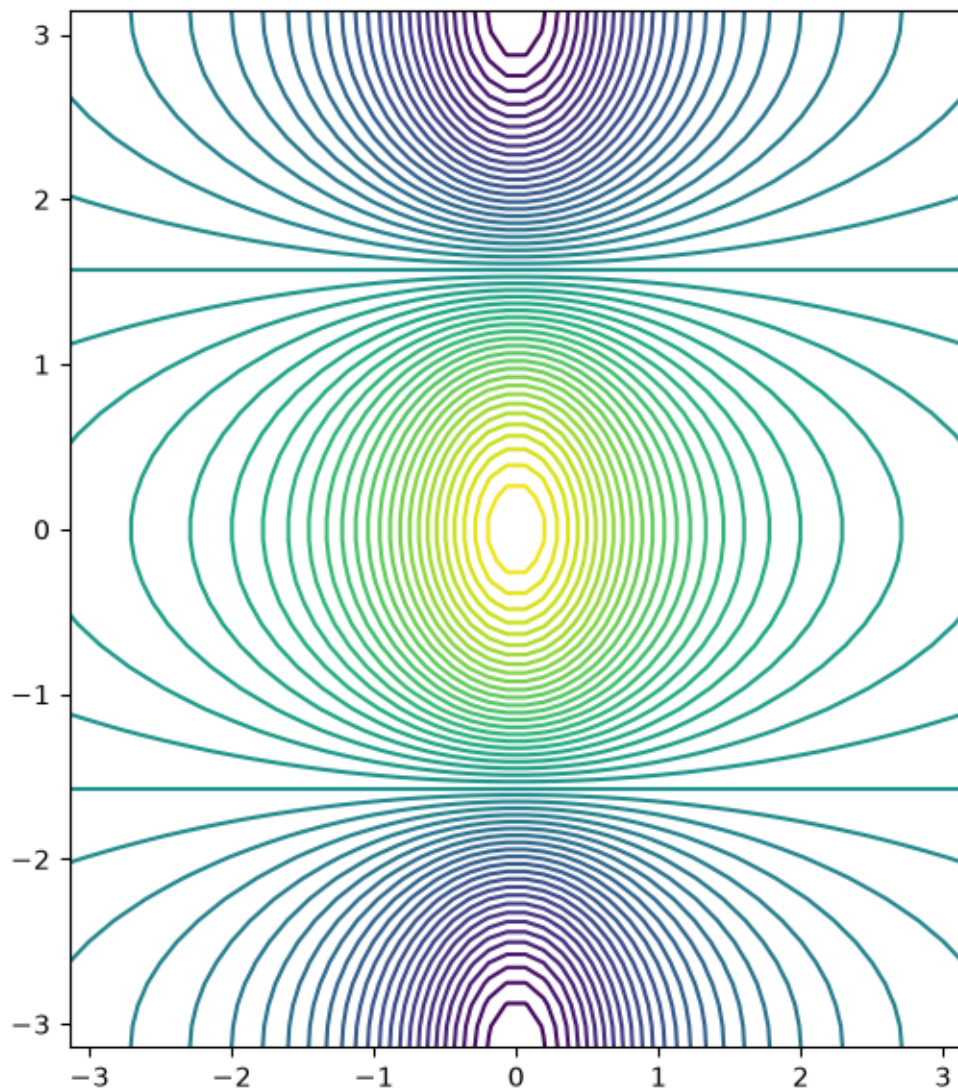
```
[83]: <matplotlib.contour.QuadContourSet at 0xffff2fecb910>
```



Dá pra aumentar a resolução aumentando o número de níveis:

```
[86]: fig, ax = subplots(figsize=(6, 7))  
      ax.contour(x, y, f, levels=50)
```

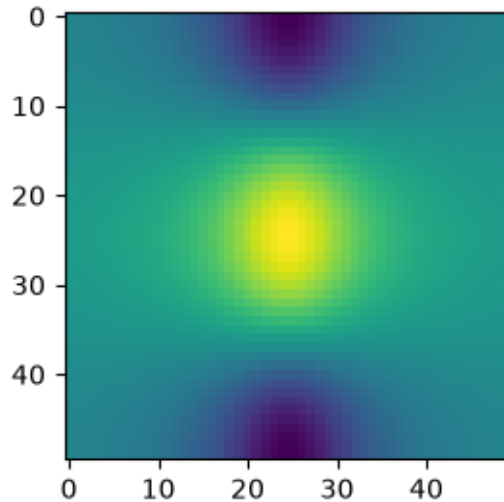
```
[86]: <matplotlib.contour.QuadContourSet at 0xffff3274ac90>
```

`ax.imshow()` é parecido com `ax.contour()`, mas produz um gráfico colorido conforme o valor de `z` — um *heatmap*, às vezes usado pra mostrar temperatura em previsão do tempo.

```
[87]: fig, ax = subplots(figsize=(3, 5))  
      ax.imshow(f)
```

```
[87]: <matplotlib.image.AxesImage at 0xffff327e6a10>
```



1.4 Sequências e notações de fatiamento, `np.linspace()` cria uma sequência de números.

```
[90]: seq1 = np.linspace(0, 12, 30)
      seq1
```

```
[90]: array([ 0.          ,  0.4137931 ,  0.82758621,  1.24137931,  1.65517241,
            2.06896552,  2.48275862,  2.89655172,  3.31034483,  3.72413793,
            4.13793103,  4.55172414,  4.96551724,  5.37931034,  5.79310345,
            6.20689655,  6.62068966,  7.03448276,  7.44827586,  7.86206897,
            8.27586207,  8.68965517,  9.10344828,  9.51724138,  9.93103448,
            10.34482759, 10.75862069, 11.17241379, 11.5862069 , 12.          ])
```

`np.arange()` devolve uma sequência espaçada por `step` (padrão 1). Vamos criar uma sequência de 0 a 20:

```
[91]: seq2 = np.arange(0, 20)
      seq2
```

```
[91]: array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
            17, 18, 19])
```

Por que o 10 não aparece? Tem a ver com a notação de *slice* do Python, usada pra indexar sequências (listas, tuplas, arrays). Pra pegar do 4º ao 6º caractere de uma string (inclusive), usamos `[0:3]`.

```
[95]: 'metodos numericos'[0:3]
```

```
[95]: 'met'
```

3:6 dentro de `[]` é um atalho pra `slice(3,6)`:

```
[96]: 'metodos numericos'[slice(3, 6)]
```

```
[96]: 'odo'
```

1.5 Indexação de dados

```
[100]: A = np.array(np.arange(36)).reshape((6, 6))  
A
```

```
[100]: array([[ 0,  1,  2,  3,  4,  5],  
             [ 6,  7,  8,  9, 10, 11],  
             [12, 13, 14, 15, 16, 17],  
             [18, 19, 20, 21, 22, 23],  
             [24, 25, 26, 27, 28, 29],  
             [30, 31, 32, 33, 34, 35]])
```

`A[2,3]` pega o elemento da segunda linha, terceira coluna (lembrando: Python indexa do 0).

```
[102]: A[2, 3]
```

```
[102]: np.int64(15)
```

O primeiro número depois do `[` se refere à linha, o segundo à coluna. `### Indexing Rows, Columns, and Submatrices` Para selecionar várias linhas de uma vez, passamos uma lista. `[2,3]` pega a segunda e a quarta linha:

```
[105]: A[[2, 3]]
```

```
[105]: array([[12, 13, 14, 15, 16, 17],  
             [18, 19, 20, 21, 22, 23]])
```

Pra selecionar a segunda e a quarta coluna, passamos `[1,3]` como segundo argumento — e usamos `:` como primeiro argumento pra pegar todas as linhas.

```
[107]: A[:, [1, 3]]
```

```
[107]: array([[ 1,  3],  
             [ 7,  9],  
             [13, 15],  
             [19, 21],  
             [25, 27],  
             [31, 33]])
```

Agora, suponha que queremos a submatriz da segunda e quarta linha, junto com a primeira e terceira coluna. Aqui a indexação fica um pouco traiçoeira — é natural tentar assim:

```
[108]: A[[1, 3], [0, 2]]
```

```
[108]: array([ 6, 20])
```

O que aconteceu? A gente recebeu um array unidimensional de tamanho 2, igual a:

```
[109]: np.array([A[1, 0], A[3, 2]])
```

```
[109]: array([ 6, 20])
```

Da mesma forma, o código abaixo também não extrai a submatriz que a gente queria (segunda/quarta linha, primeira/terceira/quarta coluna):

```
[110]: A[[1, 3], [0, 2, 3]]
```

```
-----
IndexError                                Traceback (most recent call last)
Cell In[110], line 1
----> 1 A[[1, 3], [0, 2, 3]]

IndexError: shape mismatch: indexing arrays could not be broadcast together with
↪ shapes (2,) (3,)
```

O que houve: quando passamos duas listas de índice, o numpy entende que são pares (i,j) de posições individuais — por isso as duas listas precisam ter o mesmo tamanho. Só que não era isso que a gente queria, já que buscávamos uma submatriz. Um jeito simples de fazer certo: primeiro criar uma submatriz filtrando as linhas de A, e depois, em cima dessa, filtrar as colunas.

```
[111]: A[[1, 3]][:, [0, 2]]
```

```
[111]: array([[ 6,  8],
            [18, 20]])
```

Existem jeitos mais eficientes de chegar no mesmo resultado. A função `np.ix_()` permite extrair uma submatriz usando listas, criando um objeto de mesh intermediário.

```
[115]: idx = np.ix_([2, 1], [2, 4, 5])
       A[idx]
```

```
[115]: array([[14, 16, 17],
            [ 8, 10, 11]])
```

Também dá pra selecionar submatrizes de forma eficiente usando *slices*. `1:2:1` pega o segundo e o quarto item de uma sequência, e `0:3:2` pega o primeiro e o terceiro (o terceiro número do slice é o passo).

```
[116]: A[1:2:1, 0:3:2]
```

```
[116]: array([[6, 8]])
```

Por que dá pra pegar a submatriz direto com *slices*, mas não com listas? São tipos diferentes de Python, tratados de forma diferente pelo numpy. Slices funcionam em qualquer sequência (strings, listas, tuplas); listas como índice são mais limitadas. **Boolean Indexing** Um booleano é um tipo que vale `True` ou `False` (também representado como 1 e 0). A linha abaixo cria um vetor de `False`, do tamanho da primeira dimensão de A.

```
[118]: keep_rows = np.zeros(A.shape[1], bool)
keep_rows
```

```
[118]: array([False, False, False, False, False, False])
```

Agora colocamos dois desses elementos como True:

```
[120]: keep_rows[[0, 2]] = True
keep_rows
```

```
[120]: array([ True,  True,  True,  True, False, False])
```

Os elementos de `keep_rows`, vistos como inteiros, são iguais a `np.array([0,1,0,1])`. Usamos `==` pra conferir isso — aplicado a dois arrays, `==` roda elemento por elemento.

```
[124]: np.all(np.isin(keep_rows, [2, 3, 1, 4]))
```

```
[124]: np.False_
```

(`np.all()` conferiu se todos os elementos de um array são True. Existe também `np.any()`, que confere se algum elemento é True.) Só que, mesmo `np.array([0,1,0,1])` e `keep_rows` sendo iguais pelo `==`, eles indexam linhas diferentes! O primeiro pega a primeira, segunda, primeira e segunda linha de A.

```
[127]: A[np.array([0, 1, 0, 1])]
```

```
[127]: array([[ 0,  1,  2,  3,  4,  5],
           [ 6,  7,  8,  9, 10, 11],
           [ 0,  1,  2,  3,  4,  5],
           [ 6,  7,  8,  9, 10, 11]])
```

Já `keep_rows` pega só a segunda e a quarta linha — as linhas onde o booleano é True.

```
[128]: A[keep_rows]
```

```
[128]: array([[ 0,  1,  2,  3,  4,  5],
           [ 6,  7,  8,  9, 10, 11],
           [12, 13, 14, 15, 16, 17],
           [18, 19, 20, 21, 22, 23]])
```

Isso mostra que booleanos e inteiros são tratados de forma diferente pelo numpy. De novo usando `np.ix_()`, agora criamos um mesh com a segunda e quarta linha e a primeira, terceira e quarta coluna — dessa vez aplicando a função a booleanos, não listas.

```
[129]: keep_cols = np.zeros(A.shape[1], bool)
keep_cols[[0, 2, 3]] = True
idx_bool = np.ix_(keep_rows, keep_cols)
A[idx_bool]
```

```
[129]: array([[ 0,  2,  3],
              [ 6,  8,  9],
              [12, 14, 15],
              [18, 20, 21]])
```

Também dá pra misturar uma lista com um array de booleanos nos argumentos de `np.ix_()`:

```
[133]: idx_mixed = np.ix_([3, 4], keep_cols)
       A[idx_mixed]
```

```
[133]: array([[18, 20, 21],
              [24, 26, 27]])
```

1.6 Carregando datasets com a biblioteca pandas

```
[134]: import pandas as pd
       Auto = pd.read_csv("dataset_regressao_limpo.csv")
       Auto
```

```
[134]:
```

	AMBIENT_TEMPERATURE	MODULE_TEMPERATURE	IRRADIATION	\
0	25.184316	22.857507		0.0
1	25.184316	22.857507		0.0
2	25.184316	22.857507		0.0
3	25.184316	22.857507		0.0
4	25.184316	22.857507		0.0
...	...	...	...	
68769	21.909288	20.427972		0.0
68770	21.909288	20.427972		0.0
68771	21.909288	20.427972		0.0
68772	21.909288	20.427972		0.0
68773	21.909288	20.427972		0.0
	DC_POWER			
0	0.0			
1	0.0			
2	0.0			
3	0.0			
4	0.0			
...	...			
68769	0.0			
68770	0.0			
68771	0.0			
68772	0.0			
68773	0.0			

```

[68774 rows x 4 columns]
```

```
[136]: Auto['AMBIENT_TEMPERATURE']
```

```
[136]: 0      25.184316
      1      25.184316
      2      25.184316
      3      25.184316
      4      25.184316
      ...
      68769    21.909288
      68770    21.909288
      68771    21.909288
      68772    21.909288
      68773    21.909288
      Name: AMBIENT_TEMPERATURE, Length: 68774, dtype: float64
```

O `dtype` dessa coluna é `object` — quer dizer que os valores foram lidos como texto. Vamos ver os valores únicos pra entender por quê:

```
[137]: np.unique(Auto['AMBIENT_TEMPERATURE'])

[137]: array([20.39850487, 20.42645467, 20.4631672 , ..., 34.98165257,
        35.10977173, 35.25248613], shape=(3157,))
```

O culpado é o valor `?`, usado pra marcar dado faltante. Pra resolver, passamos o argumento `na_values` no `pd.read_csv()` — cada `?` vira `np.nan` (not a number).

```
[138]: Auto = pd.read_csv('dataset_regressao_limpo.csv', na_values=['?'])
      Auto['AMBIENT_TEMPERATURE'].sum()
```

```
[138]: np.float64(1757761.7160981873)
```

O atributo `Auto.shape` diz que o dado tem 68774 observações (linhas) e 4 variáveis (colunas).

```
[139]: Auto.shape
```

```
[139]: (68774, 4)
```

Existem várias formas de lidar com dado faltante. Aqui, como só 5 linhas têm valores faltando, vamos simplesmente removê-las com `.dropna()`.

```
[141]: #já tinha feito uma limpeza prévia dos dados, por isso não alterou
      Auto_new = Auto.dropna()
      Auto_new.shape
```

```
[141]: (68774, 4)
```

1.6.1 Basics of Selecting Rows and Columns

`Auto.columns` mostra os nomes das variáveis.

```
[142]: Auto = Auto_new
```

Acessar linhas e colunas de um data frame é parecido, mas não idêntico, a acessar de um array. O primeiro argumento do [] sempre se aplica às linhas — um *slice* cria um data frame com as linhas correspondentes:

```
[147]: Auto[:3]
```

```
[147]:   AMBIENT_TEMPERATURE  MODULE_TEMPERATURE  IRRADIATION  \
0           25.184316           22.857507           0.0
1           25.184316           22.857507           0.0
2           25.184316           22.857507           0.0

   DC_POWER
0         0.0
1         0.0
2         0.0
```

```
[149]: # Remove espaços antes e depois do nome de todas as colunas
Auto.columns = Auto.columns.str.strip()
```

Um array de booleanos também filtra linhas:

```
[151]: idx_80 = Auto['IRRADIATION'] > 1
Auto[idx_80]
```

```
[151]:   AMBIENT_TEMPERATURE  MODULE_TEMPERATURE  IRRADIATION  DC_POWER
12082           29.597730           56.443898           1.038991  11500.00000
12083           29.597730           56.443898           1.038991  13913.57143
12084           29.597730           56.443898           1.038991  13687.12500
12085           29.597730           56.443898           1.038991  13552.62500
12086           29.597730           56.443898           1.038991  13266.25000
...
67757           28.649095           53.773537           1.031230  13627.42857
67758           28.649095           53.773537           1.031230  13692.85714
67759           28.649095           53.773537           1.031230  13287.85714
67760           28.649095           53.773537           1.031230  13586.50000
67761           28.649095           53.773537           1.031230  13301.42857
```

```
[682 rows x 4 columns]
```

Já passando uma lista de strings no [], o resultado é um data frame com aquelas colunas:

```
[152]: Auto[['MODULE_TEMPERATURE', 'IRRADIATION']]
```

```
[152]:   MODULE_TEMPERATURE  IRRADIATION
0           22.857507           0.0
1           22.857507           0.0
2           22.857507           0.0
3           22.857507           0.0
4           22.857507           0.0
```



```

...
68769      20.427972      0.0
68770      20.427972      0.0
68771      20.427972      0.0
68772      20.427972      0.0
68773      20.427972      0.0

```

[68774 rows x 2 columns]

Como não definimos uma coluna de índice ao carregar, as linhas são numeradas de 0 a 68774:

```
[153]: Auto.index
```

```
[153]: RangeIndex(start=0, stop=68774, step=1)
```

set_index() renomeia as linhas usando o conteúdo de Auto['AMBIENT_TEMPERATURE'].

```
[155]: Auto_re = Auto.set_index('AMBIENT_TEMPERATURE')
Auto_re
```

```
[155]:
```

	MODULE_TEMPERATURE	IRRADIATION	DC_POWER
AMBIENT_TEMPERATURE			
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
...	...	...	...
21.909288	20.427972	0.0	0.0
21.909288	20.427972	0.0	0.0
21.909288	20.427972	0.0	0.0
21.909288	20.427972	0.0	0.0
21.909288	20.427972	0.0	0.0

[68774 rows x 3 columns]

```
[156]: Auto_re.columns
```

```
[156]: Index(['MODULE_TEMPERATURE', 'IRRADIATION', 'DC_POWER'], dtype='object')
```

Repare que AMBIENT_TEMPERATURE não existe mais como coluna — virou o índice. Agora que o índice é o nome, dá pra acessar linhas por nome usando .loc[]:

```
[161]: rows = [Auto_re.index[0]]
Auto_re.loc[rows]
```

```
[161]:
```

	MODULE_TEMPERATURE	IRRADIATION	DC_POWER
AMBIENT_TEMPERATURE			
25.184316	22.857507	0.0	0.0

25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0

Como alternativa, `.iloc[]` pega linhas pela posição — por exemplo, a 4^a e a 5^a linha:

```
[162]: Auto_re.iloc[[3, 4]]
```

```
[162]:
```

	MODULE_TEMPERATURE	IRRADIATION	DC_POWER
AMBIENT_TEMPERATURE			
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0

Também dá pra usar pra pegar a 1^a, 2^a e 3^a coluna:

```
[164]: Auto_re.iloc[:, [0, 1, 2]]
```

```
[164]:
```

	MODULE_TEMPERATURE	IRRADIATION	DC_POWER
AMBIENT_TEMPERATURE			
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0
...	...	...	...
21.909288	20.427972	0.0	0.0
21.909288	20.427972	0.0	0.0
21.909288	20.427972	0.0	0.0
21.909288	20.427972	0.0	0.0
21.909288	20.427972	0.0	0.0

[68774 rows x 3 columns]

Dá pra pegar linhas e colunas ao mesmo tempo, num único `.iloc[]`:

```
[166]: Auto_re.iloc[[2, 3], [0, 1, 2]]
```

```
[166]:
```

	MODULE_TEMPERATURE	IRRADIATION	DC_POWER
AMBIENT_TEMPERATURE			
25.184316	22.857507	0.0	0.0
25.184316	22.857507	0.0	0.0

1.6.2 Suponha que quero um df só com os dados de quando a irradiação é maior que zero quando a temperatura do módulo é maior que 23

```
[ ]: #quero um df só com os dados de quando a coluna IRRADIATION é maior que 0 e
      ↳ AMBIENT_TEMPERATURE é maior que 20
idx_irradiation = Auto_re['IRRADIATION'] > 0
idx_ambient_temp = Auto_re['MODULE_TEMPERATURE'] > 23
Auto_re.loc[idx_irradiation & idx_ambient_temp]
```

```
[ ]:
```

	MODULE_TEMPERATURE	IRRADIATION	DC_POWER
AMBIENT_TEMPERATURE			
24.289211	23.096692	0.000863	0.000
24.289211	23.096692	0.000863	0.000
24.289211	23.096692	0.000863	0.000
24.289211	23.096692	0.000863	0.000
24.289211	23.096692	0.000863	0.000
...	...	...	...
24.038157	24.068250	0.023446	346.875
24.038157	24.068250	0.023446	337.500
24.038157	24.068250	0.023446	339.500
24.038157	24.068250	0.023446	332.625
24.038157	24.068250	0.023446	340.250

[34886 rows x 3 columns]

Pra fazer isso de forma mais concisa, usamos uma função anônima, a `lambda`:

```
[169]: Auto_re.loc[lambda df: (df['IRRADIATION'] > 0) & (df['MODULE_TEMPERATURE'] >
      ↳ 23)]
```

```
[169]:
```

	MODULE_TEMPERATURE	IRRADIATION	DC_POWER
AMBIENT_TEMPERATURE			
24.289211	23.096692	0.000863	0.000
24.289211	23.096692	0.000863	0.000
24.289211	23.096692	0.000863	0.000
24.289211	23.096692	0.000863	0.000
24.289211	23.096692	0.000863	0.000

...	...	...	...
24.038157	24.068250	0.023446	346.875
24.038157	24.068250	0.023446	337.500
24.038157	24.068250	0.023446	339.500
24.038157	24.068250	0.023446	332.625
24.038157	24.068250	0.023446	340.250

[34886 rows x 3 columns]

& faz um “e” elemento a elemento.

```
[175]: Auto_re.loc[lambda df: (df['IRRADIATION'] > 0) & (df['MODULE_TEMPERATURE'] > 23) | (df['DC_POWER'] > 20)]
```

	MODULE_TEMPERATURE	IRRADIATION	DC_POWER
AMBIENT_TEMPERATURE			
24.289211	23.096692	0.000863	0.000000
24.289211	23.096692	0.000863	0.000000
24.289211	23.096692	0.000863	0.000000
24.289211	23.096692	0.000863	0.000000
24.289211	23.096692	0.000863	0.000000
...	...	...	...
23.840239	22.968658	0.007007	79.000000
23.840239	22.968658	0.007007	75.714286
23.840239	22.968658	0.007007	77.428571
23.840239	22.968658	0.007007	74.857143
23.840239	22.968658	0.007007	77.714286

[37496 rows x 3 columns]

| faz um “ou” elemento a elemento.

1.7 Loop FOR

```
[176]: total = 0
for value in [4, 10, 20]:
    total += value
print('Total is: {0}'.format(total))
```

Total is: 34

O código indentado abaixo do `for` roda pra cada valor da sequência. O loop termina quando a célula acaba ou quando o código volta a ficar no mesmo nível de indentação do `for`. Dá pra aninhar loops com indentação extra.

```
[177]: total = 0
for value in [4, 10, 20]:
    for weight in [5, 20, 10]:
        total += value * weight
```

```
print('Total is: {0}'.format(total))
```

Total is: 1190

Aqui somamos cada combinação de `value` e `weight`. Também usamos a notação de incremento: `a += b` equivale a `a = a + b`. Uma tarefa mais comum é somar pares (`value`, `weight`) — por exemplo, calcular a média de uma variável que assume valores 2, 3 ou 19 com probabilidade 0.2, 0.3, 0.5. Pra isso dá pra usar `zip()`, que percorre uma sequência de tuplas.

```
[178]: total = 0
for value, weight in zip([4, 10, 20], [5, 20, 10]):
    total += weight * value
print('Weighted average is: {0}'.format(total))
```

Weighted average is: 420

1.7.1 Formatação de string

```
[182]: rng = np.random.default_rng(1)
A = rng.standard_normal((10, 6))
M = rng.choice([6, np.nan], p=[0.7, 0.3], size=A.shape)
A += M
D = pd.DataFrame(A, columns=['alfa', 'bravo', 'charlie', 'delta', 'echo', 'foxtrot'])
D[:3]
```

```
[182]:      alfa      bravo  charlie      delta      echo      foxtrot
0  6.345584  6.821618      NaN      NaN  6.905356  6.446375
1  5.463047      NaN      NaN  6.294132  6.028422      NaN
2  5.263546  5.837090  5.517881  6.598846      NaN      NaN
```

```
[183]: for col in D.columns:
    template = 'Column "{0}" has {1:.2%} missing values'
    print(template.format(col, np.isnan(D[col]).mean()))
```

Column "foxtrot" has 50.00% missing values

O método `template.format()` espera dois argumentos, `{0}` e `{1:.2%}` — o segundo já vem com formatação, indicando que deve aparecer como porcentagem com duas casas decimais.

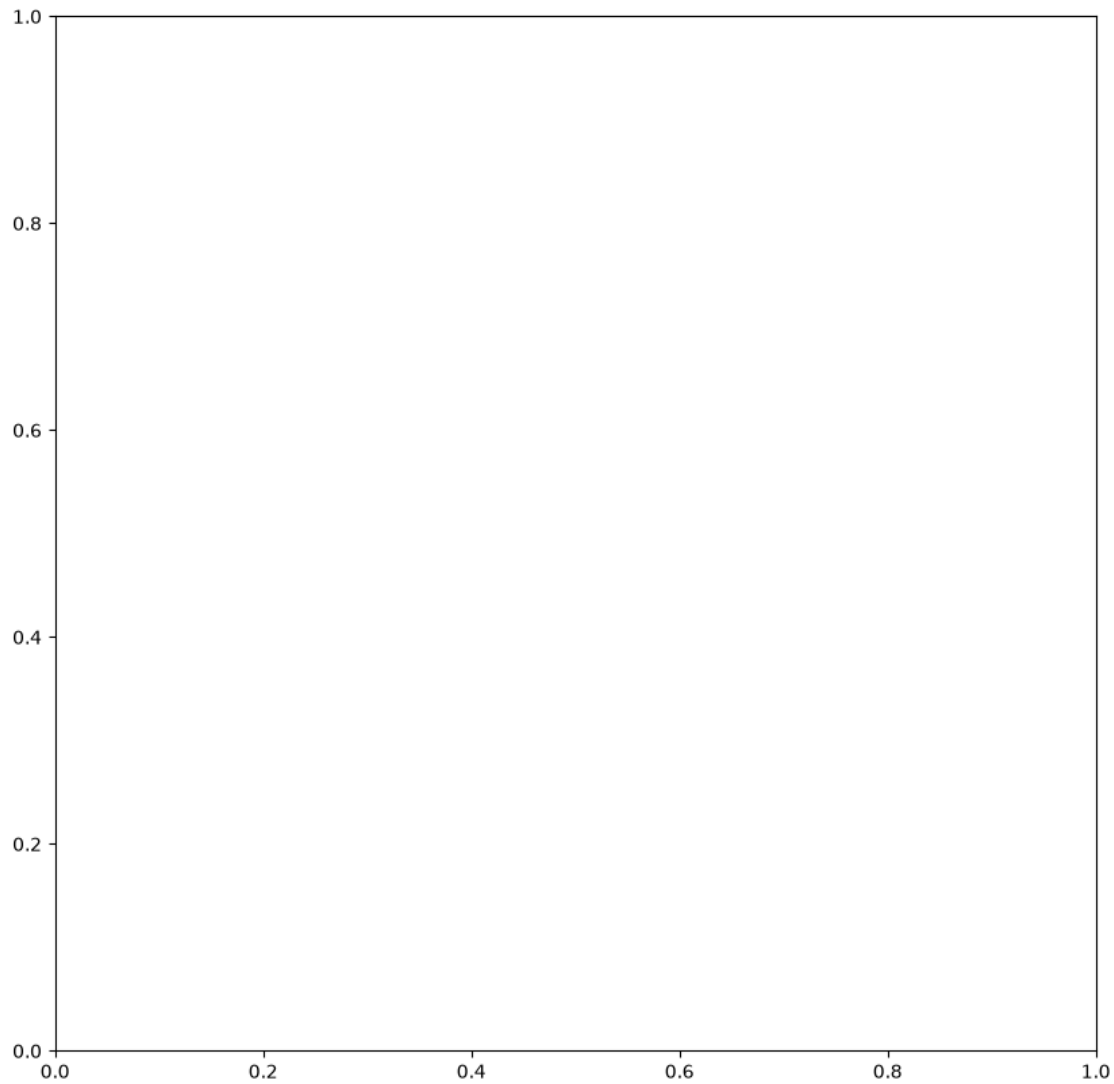
1.8 Resumos Gráficos e Numéricos Adicionais.

1.8.1 Dá pra usar `ax.plot()` ou `ax.scatter()` pra ver variáveis quantitativas. Só que digitar o nome da variável direto dá erro, porque o Python não sabe que deve procurar dentro do dataset `Auto`.

```
[184]: fig, ax = subplots(figsize=(10, 10))
ax.plot(DC_POWER, mpg, 'o')
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[184], line 2  
      1 fig, ax = subplots(figsize=(10, 10))  
----> 2 ax.plot(DC_POWER, mpg, 'o')
```

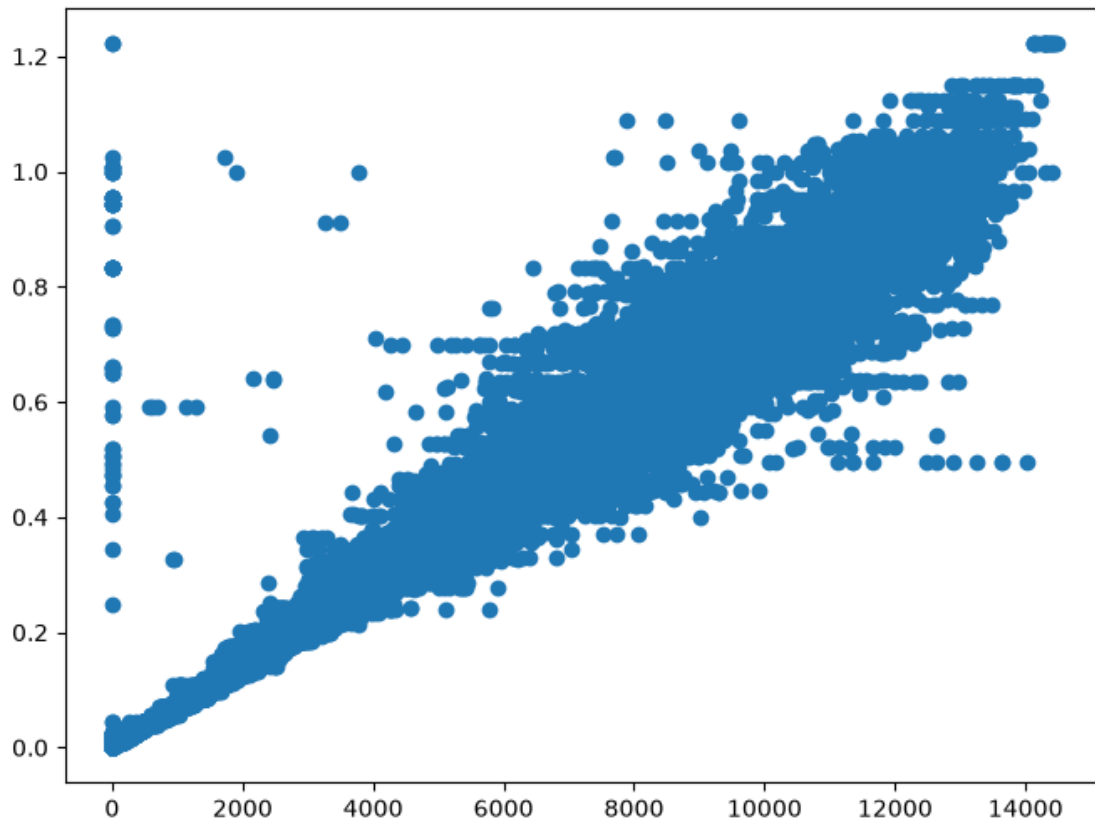
NameError: name 'DC_POWER' is not defined



O jeito certo é acessar a coluna direto:

```
[188]: fig, ax = subplots(figsize=(8, 6))  
       ax.plot(Auto['DC_POWER'], Auto['IRRADIATION'], 'o')
```

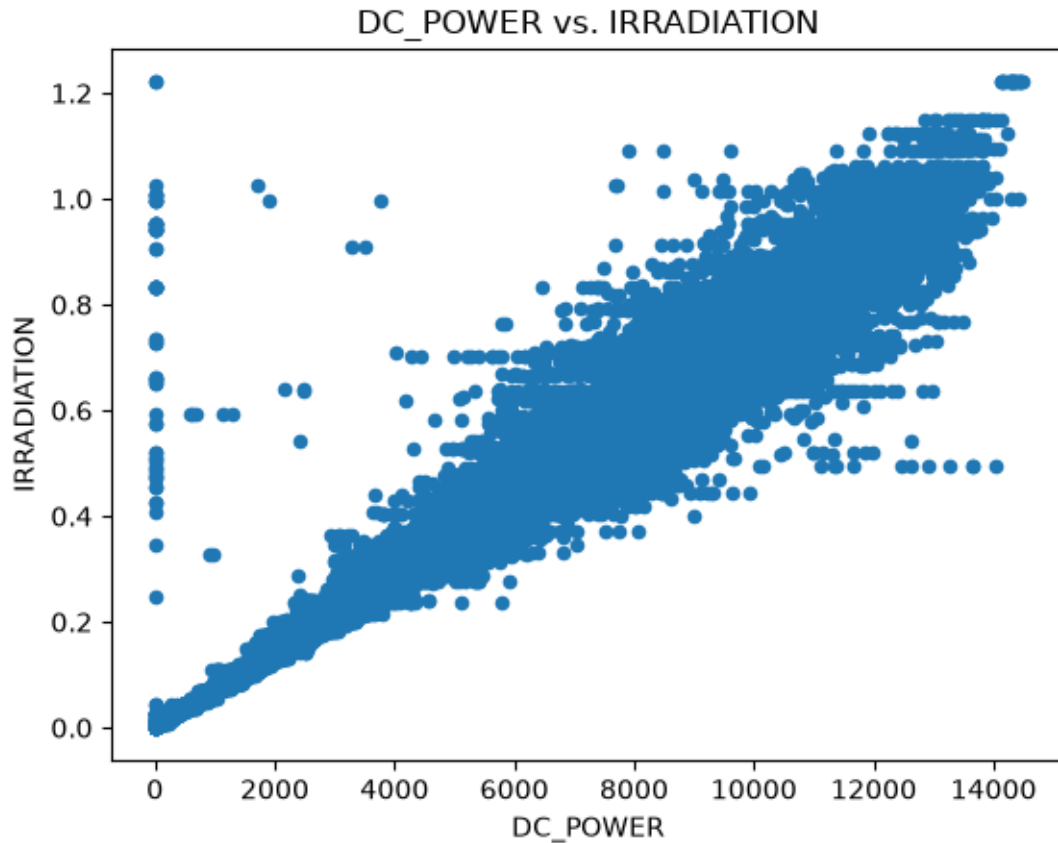
```
[188]: [<matplotlib.lines.Line2D at 0xffff27562d50>]
```



Como alternativa, dá pra usar o método `.plot()` do próprio data frame — aí sim as variáveis podem ser acessadas pelo nome. O `.plot()` de um data frame devolve um `axes`, que dá pra usar pra atualizar o gráfico como antes.

```
[189]: ax = Auto.plot.scatter('DC_POWER', 'IRRADIATION')
      ax.set_title('DC_POWER vs. IRRADIATION')
```

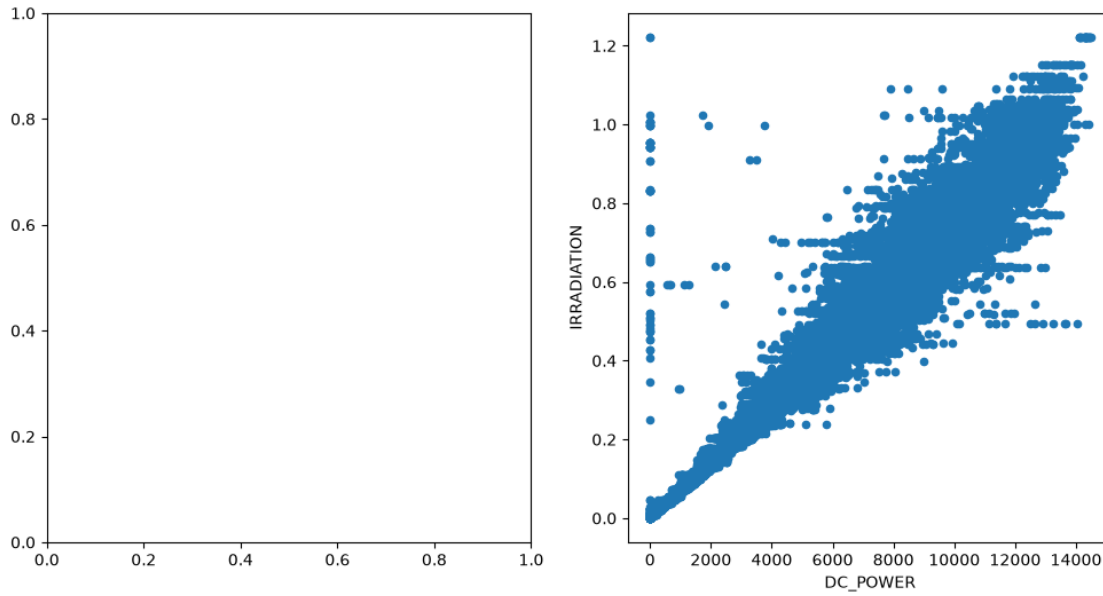
```
[189]: Text(0.5, 1.0, 'DC_POWER vs. IRRADIATION')
```



Dá pra mandar o data frame plotar num `axes` específico. Nesse caso o método devolve o mesmo `axes` que passamos como argumento. Repare que, ao pedir uma grade unidimensional de plots, o objeto `axes` também é unidimensional. Colocamos o scatter no gráfico do meio, de uma fileira de três.

```
[191]: fig, axes = subplots(ncols=2, figsize=(12, 6))  
Auto.plot.scatter('DC_POWER', 'IRRADIATION', ax=axes[1])
```

```
[191]: <Axes: xlabel='DC_POWER', ylabel='IRRADIATION'>
```

As colunas de um data frame também podem ser acessadas como atributo: tente digitar `Auto.horsepower`. Agora a variável `cylinders`. Digitando `Auto.cylinders.dtype` vemos que ela é tratada como quantitativa — mas como só existem poucos valores possíveis, faz mais sentido tratar como qualitativa. Abaixo, substituímos a coluna `cylinders` por uma versão categórica dela.

```
[194]: Auto.cylinders = pd.Series(Auto.DC_POWER, dtype='category')
Auto.cylinders.dtype
```

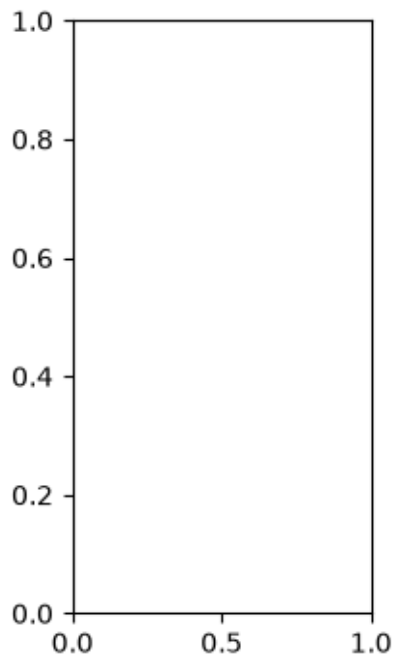
```
/tmp/ipykernel_1857/3316582091.py:1: UserWarning: Pandas doesn't allow columns
to be created via a new attribute name - see https://pandas.pydata.org/pandas-
docs/stable/indexing.html#attribute-access
```

```
Auto.cylinders = pd.Series(Auto.DC_POWER, dtype='category')
```

```
[194]: CategoricalDtype(categories=[          0.0,          8.75,          8.875,          9.0,
          9.25,          9.5,          9.625,          10.0,
          10.25,          10.625,
          ...,
          14329.14286, 14351.125, 14357.42857, 14370.5,
          14413.42857, 14416.14286, 14418.42857, 14436.28571,
          14466.85714, 14471.125],
, ordered=False, categories_dtype=float64)
```

Agora que `cylinders` é qualitativa, dá pra mostrar com um `boxplot()`.

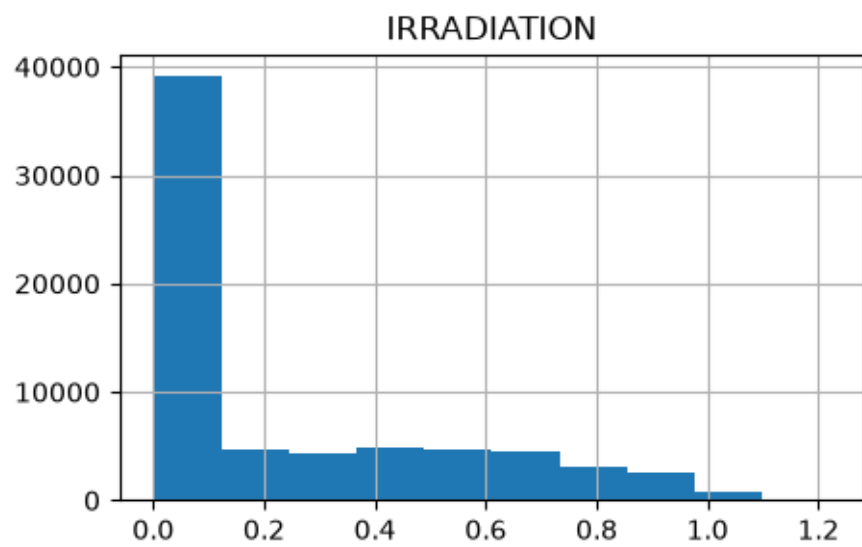
```
[200]: fig, ax = subplots(figsize=(2, 4))
```



O método `hist()` plota um histograma.

```
[198]: fig, ax = subplots(figsize=(5, 3))  
Auto.hist('IRRADIATION', ax=ax)
```

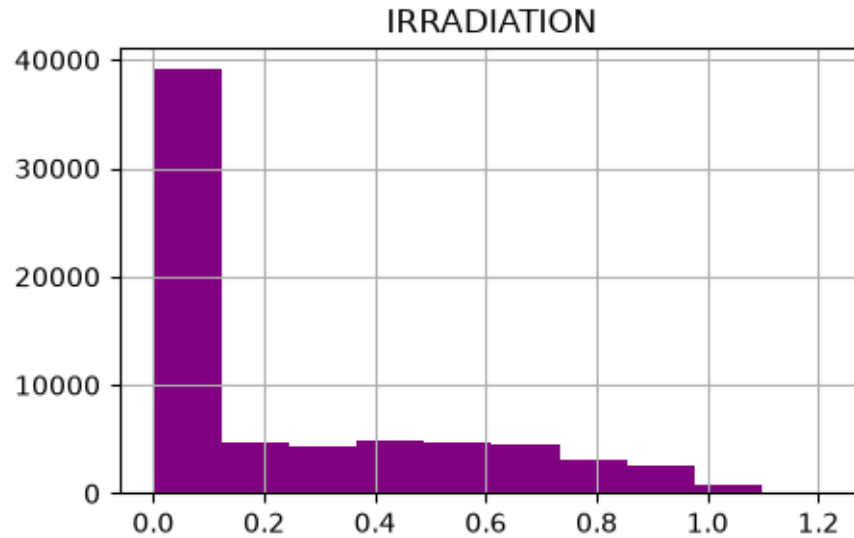
```
[198]: array([<Axes: title={'center': 'IRRADIATION'}>], dtype=object)
```



A cor das barras e o número de *bins* dá pra mudar:

```
[206]: fig, ax = subplots(figsize=(5, 3))
Auto.hist('IRRADIATION', color='purple', bins=10, ax=ax)
```

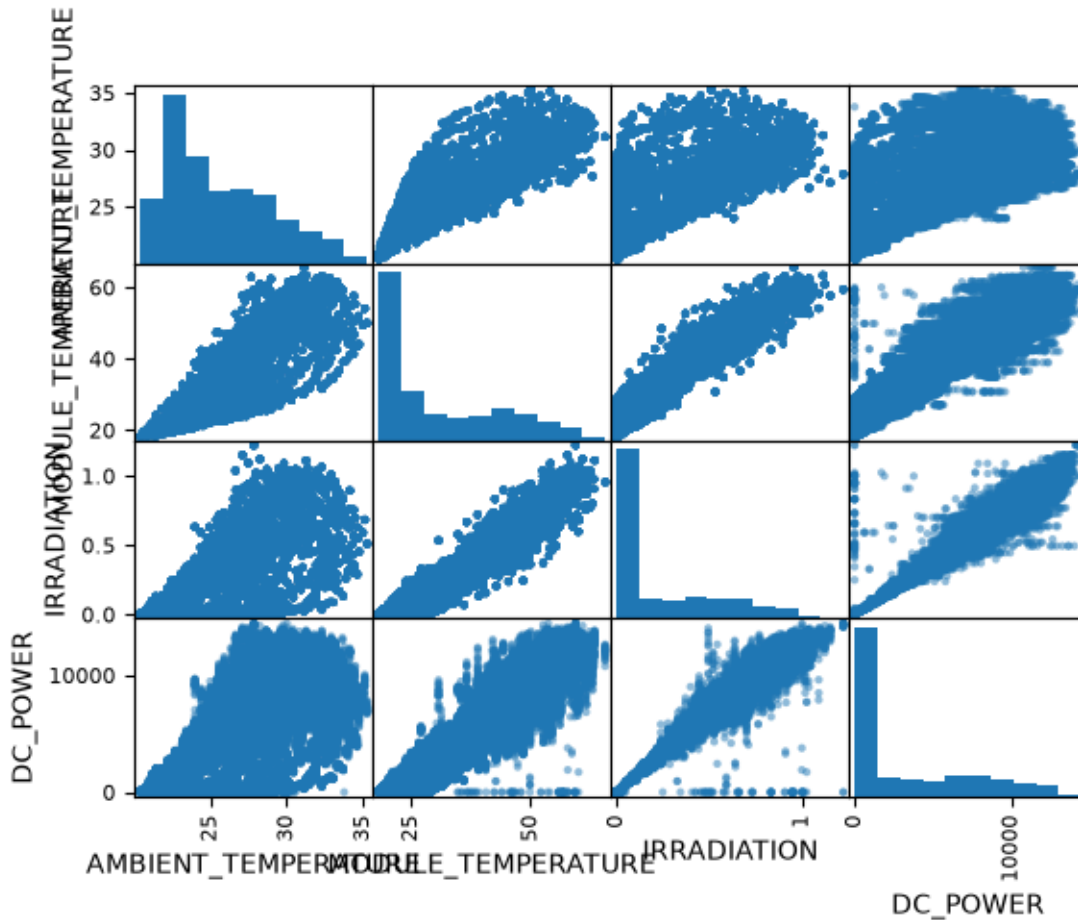
```
[206]: array([<Axes: title={'center': 'IRRADIATION'}>], dtype=object)
```



`pd.plotting.scatter_matrix()` cria uma matriz de gráficos de dispersão, mostrando todas as relações par a par entre as colunas de um data frame.

```
[202]: pd.plotting.scatter_matrix(Auto)
```

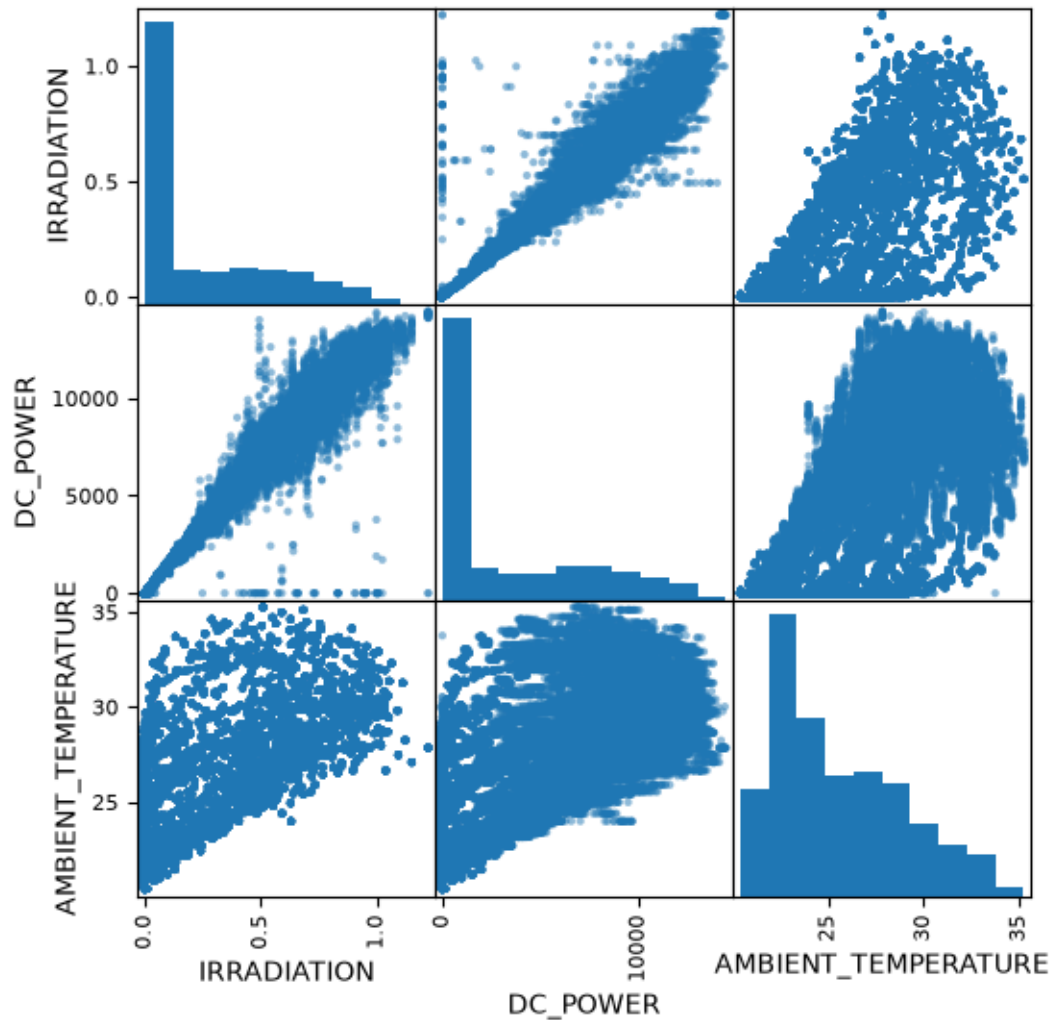
```
[202]: array([[<Axes: xlabel='AMBIENT_TEMPERATURE', ylabel='AMBIENT_TEMPERATURE'>,
  <Axes: xlabel='MODULE_TEMPERATURE', ylabel='AMBIENT_TEMPERATURE'>,
  <Axes: xlabel='IRRADIATION', ylabel='AMBIENT_TEMPERATURE'>,
  <Axes: xlabel='DC_POWER', ylabel='AMBIENT_TEMPERATURE'>],
 [<Axes: xlabel='AMBIENT_TEMPERATURE', ylabel='MODULE_TEMPERATURE'>,
  <Axes: xlabel='MODULE_TEMPERATURE', ylabel='MODULE_TEMPERATURE'>,
  <Axes: xlabel='IRRADIATION', ylabel='MODULE_TEMPERATURE'>,
  <Axes: xlabel='DC_POWER', ylabel='MODULE_TEMPERATURE'>],
 [<Axes: xlabel='AMBIENT_TEMPERATURE', ylabel='IRRADIATION'>,
  <Axes: xlabel='MODULE_TEMPERATURE', ylabel='IRRADIATION'>,
  <Axes: xlabel='IRRADIATION', ylabel='IRRADIATION'>,
  <Axes: xlabel='DC_POWER', ylabel='IRRADIATION'>],
 [<Axes: xlabel='AMBIENT_TEMPERATURE', ylabel='DC_POWER'>,
  <Axes: xlabel='MODULE_TEMPERATURE', ylabel='DC_POWER'>,
  <Axes: xlabel='IRRADIATION', ylabel='DC_POWER'>,
  <Axes: xlabel='DC_POWER', ylabel='DC_POWER'>]], dtype=object)
```



Também dá pra fazer só pra um subconjunto de variáveis:

```
[203]: pd.plotting.scatter_matrix(Auto[['IRRADIATION', 'DC_POWER',
↪ 'AMBIENT_TEMPERATURE']], figsize=(6, 6))
```

```
[203]: array([[<Axes: xlabel='IRRADIATION', ylabel='IRRADIATION'>,
<Axes: xlabel='DC_POWER', ylabel='IRRADIATION'>,
<Axes: xlabel='AMBIENT_TEMPERATURE', ylabel='IRRADIATION'>],
[<Axes: xlabel='IRRADIATION', ylabel='DC_POWER'>,
<Axes: xlabel='DC_POWER', ylabel='DC_POWER'>,
<Axes: xlabel='AMBIENT_TEMPERATURE', ylabel='DC_POWER'>],
[<Axes: xlabel='IRRADIATION', ylabel='AMBIENT_TEMPERATURE'>,
<Axes: xlabel='DC_POWER', ylabel='AMBIENT_TEMPERATURE'>,
<Axes: xlabel='AMBIENT_TEMPERATURE', ylabel='AMBIENT_TEMPERATURE'>]],
dtype=object)
```



O método `describe()` gera um resumo numérico de cada coluna de um data frame.

```
[204]: Auto[['DC_POWER', 'IRRADIATION']].describe()
```

```
[204]:
```

	DC_POWER	IRRADIATION
count	68774.000000	68774.000000
mean	3147.177450	0.232305
std	4036.441826	0.301948
min	0.000000	0.000000
25%	0.000000	0.000000
50%	428.571429	0.031620
75%	6365.468750	0.454880
max	14471.125000	1.221652

Também dá pra pedir o resumo de uma única coluna:

```
[205]: Auto['IRRADIATION'].describe()
```

```
[205]: count      68774.000000  
      mean         0.232305  
      std          0.301948  
      min          0.000000  
      25%          0.000000  
      50%          0.031620  
      75%          0.454880  
      max          1.221652  
      Name: IRRADIATION, dtype: float64
```

Pra sair do Jupyter, vá em File / Close and Halt.